- Operational Work
- Pulse Guide
- Work with Rules

On this page

Work with Rules

Learn how to fully manage your rules, including:

- Understanding how rules work.
- Writing rules that are specific to your business.
- Evaluating and testing rules.
- Deploying rules to runtime.

As a data scientist or fraud prevention agent, you can leverage rules to detect and mitigate fraud events at scale in real time. The following user journey shows the different stages required to set up your rules and deploy them to runtime:

Understand rulesâ

This section provides you with an overview of how rules work to detect and prevent fraud with the resources that come out of the box with your application.

API response with Feedzai's recommendationââ

When a PSP invokes Feedzai's API to score a request via the payments endpoint, or merchant provisioning, Feedzai sends an API response containing a decision recommendation. This field helps you decide whether the payment is fraudulent or legitimate:

The event is processed by the risk engine platform, Pulse, which uses data science resources to evaluate the event. If you look at the result generated by rules, the API response contains a decision field with the recommendation values `approve` or `decline`, and the boolean alert field. The following section explains how both decision outcomes and alerts are generated.

How is Feedzai's recommendation generated?ââ

It is important to understand how rules process events and contribute to the API response to detect and prevent fraud efficiently. This section breaks down this life cycle in the following conceptual steps:

1. Mapping API fields to schema fields
2. Rule options in the Pulse application
3. Rule blocks and outcomes in the Pulse workflow
4. Manual revision in Case Manager
5. API response

1. Mapping API fields to Pulse schemaââ

The fields in the API request are mapped to a Pulse schema named Payments API Schema. Learn more about data schemas to better grasp what they are used for.

2. Rule options in the Pulse applicationââ

Before getting into the specific logic behind each event, let's have a closer look at two important rule settings. Each rule is configured with Actions and Mark As:

When a rule is triggered, Pulse sends the event to Case Manager as "Alerted" for manual revision. This also sets the alert field as `true` in the API response, thus allowing fraud prevention agents to focus their investigation only on suspicious activity.

- The **Actions** option includes the rule code (e.g. Device-ID-Review-PN13) so that it is possible to identify, in the API response's **Actions** array, which rules triggered.

- The **Mark As** option lets you select the outcome values that are configured beforehand in the Pulse workflow. When a rule is triggered, the **Mark As** value is written to the schema field that is associated with this rule outcome. For example, in this case, the `decline` value is written in the decisionâs internal field.

Different rules are defined with different outcomes according to the specificities of each event. The position of rules projects in the Pulse workflow, and the additional custom services dictate how the final recommendation is generated. This is further discussed in the next step.

3. Rule blocks and outcomes in the Pulse workflowââ

Each set of APIs contains a Pulse workflow (i.e. *Merchant Provisioning Workflow*, *Payments Workflow*, and *Reference Data Workflow*). The possible rule outcomes and the rules' processing order are defined in each workflow:

Payments workflow

How does the *Payments Workflow* generate the final outcome?

The Payments Workflow is configured with decision and fraud outcomes. The fraud outcome is generally configured to be used by machine learning models. This means that each rule contributes to the decision field, and the first rule triggering sets the final value.

4. Manual revision in Case Managerâ□□â□□

Payment events can generate alerts when one of the rules, configured with the Alert action, is triggered. The event is then displayed in Case Managerâ□□s Alerts page.

Fraud prevention agents investigate alerts in Case Manager and decide whether the event is fraudulent or legitimate. This decision is registered in the `feedback_value` field as `true` or `false` for `fraud` and `not fraud` (`feedback_label`), respectively. Then, the Feedback Rules project evaluates this result and makes a decision on adding entities, such as Card ID, to PosNeg Lists. The following payments can then be automatically `approved` or `declined` .

5. API responseâ□□â□□

At this point, you already understand how Feedzai scores an event. Note that, when you send an API request via Payments stream using one of its scoring endpoints, you will get an API response with the following key fields:

- `decision` : The final outcome generated by rules.
- `alert` : Whether the event generated an alert that is visible on the Alerts page in Case Manager.
- `score` : The numeric value a machine learning model generates to evaluate an event. Possible values range from 0 to 1000.
- `action_codes` : Every rule has an array of Actions including the name of the rule (e.g. [Device-IP-PN1]). The `action_codes` array includes all the actions of triggered rules to further understand which rules have triggered for each event.

Rule configurations in Pulseâ□□â□□

To define a rule in Pulse, you have the following main configurations available:

- **Variables**: Allows you to declare variables that are further used in the rule's condition including thresholds or transformation/conversion of the input value (e.g. variable that converts an amount from cents to US dollar). Variables are useful to avoid having large condition statements, which are error-prone and difficult to maintain.
- **Conditions**: The condition expression must return a boolean value that dictates whether the event triggers a rule.
- **Explanations**: Allows you to understand why a rule has triggered in a human-readable way. If an event triggers, the explanation is displayed in Case Manager to provide additional insights to fraud prevention agents.
- **Actions**: Operations that are performed when the rule is triggered. The standard values that help you meet your custom business logic are:
    - - Sends the event to Case Manager's Alerts page for manual revision.
    - - Flags entities such as cards, accounts, devices, among others, to be added to the PosNeg list with the decline value.
    - - Flags entities such as cards, accounts, devices, among others, to be added to the PosNeg list with the approve value.
- **Decisions**: The rule decision on the outcome of the transaction. The standard values that help you meet your custom business logic are:
    - The decision of this rule is to approve the event.
    - The decision of this rule is to decline the event.
    - This rule does not decide the event's outcome.

Custom business logicâ□□â□□

The combination of rule actions and decisions described in the previous section allows you to easily define custom business logic. For example, imagine that you want to define the transaction responses:

- **Soft decline**: This use case involves rules that decline the transaction but also generate an alert. The transaction should be displayed in Case Managerâ□□s Alert page for fraud prevention agent review.
- **Hard decline**: This use case involves rules that decline the transaction and do not generate an alert. The transaction wonâ□□t be displayed in Case Managerâ□□s Alert page for fraud prevention agent review.

- **Soft approve**: This use case involves rules that approve the transaction but also generate an alert. The transaction should be displayed in Case Managerâ□□s Alert page for fraud prevention agent review.
- **Hard approve**: This use case involves rules that approve the transaction and do not generate an alert. The transaction wonâ□□t be displayed in Case Managerâ□□s Alert page for fraud prevention agent review.

Understand rules projectsâ□□

A rules project is an ordered collection of rules that share similar traits. Grouping your rules in different rules projects allows you to:

- Keep your work organized.
- Set general configurations (e.g. a schema) that apply to all the rules that share similarities.
- Order rules projects in a workflow so that real-time events are first processed by rules with higher priority and then by rules with lower priority.

Standard rules projectsâ□□â□□

Transaction Fraud for PSPs includes standard rules projects to help you get started:

(*) Fraud Rules are subdivided into several logics, i.e. velocity rules, behavior rules, MCC rules, and mismatch rules.

Conceptually, rules projects are divided into the following main groups according to the event type that arrives to the workflow:

- Feedback events
- Scorable events

Feedback eventsâ□□â□□

Feedback events are evaluated by the following rules project:

- **Feedback Rules**: Contains rules that classify entities into â□□declineâ□□or â□□approveâ□□lists according to the fraud prevention agent's feedback on the event. Examples of rules include:
    - Feedback fraud: Entity is listed to be declined if the fraud prevention agent marks it as "Fraud", or if there is a Feedback event received through the API marking the event as "Fraud".
    - Feedback not fraud: Entity is listed to be approved if the fraud prevention agent marks it as "Not Fraud", or if there is a Feedback event received through the API marking the event as "Not Fraud".

The typical flow of a triggered feedback rule is as follows:

To learn how Feedback rules automatically put an entity into a PosNeg list, check the following sections:

- Create lists
- List management service

Scorable eventsâ□□â□□

Scorable events are the events that return a decision outcome. For instance, Payment events, or particular Info events, such as Refunds or OCTs, return a decision outcome. Scorable events are evaluated by the following rules projects:

- **APM Rules**, **Card Present Rules** and **Fraud Rules**: Contains rules that look for fraud patterns.
- **By Merchant PosNeg Rules**: Implements decisions based on listed entity values. This means that these rules will check if entities are listed to be approved / declined, and, based on that, they set the respective approve / decline decision. Note that, for PosNeg rules, you should use complex lists.
- **PosNeg Rules**: Implements decisions based on listed entity values. This means that these rules will check if entities are listed to be approved / declined, and, based on that, they set the respective approve / decline decision. Note that, for PosNeg rules, you should use complex lists.
- **Refund/OCT Rules**: Contains rules that look for fraudulent refunds / OCTs.
- **Decision Rules**: Contains rules that have triggered based on the score produced by a data science model.

All eventsââ

The following rules project is evaluated for all event types:

- **Self-service Rules**: Contains rules that are written in Case Manager by Fraud Prevention Managers and are processed in Pulse. Fraud Prevention Managers can use this functionality to write global and âby Merchantâ rules directly in production to quickly act on an emerging fraud pattern.
- **By Merchant Self-service Rules**: Contains rules that are written in Case Manager by Fraud Prevention Agents (merchant) and are processed in Pulse. There rules are shared between the fraud prevention agent (merchant) and the fraud prevention manager (PSP) to write âby Merchantârules directly in production to quickly act on an emerging fraud pattern.

Rules visibility in Case Managerâ

The visibility of rules in Case Manager can be managed in the respective rules project.

By default, the visibility will be public, which means that the triggered rules and respective explanations are available to all users that have generic permissions to see rules.

If the rules of a specific rules project must be visible to the landlord, perform the following configuration:

1. In the sidebar, open the **Data Science** tab.
2. Scroll down to find the Rules Projects widget.
3. Select the **Configure** icon under Actions to edit the respective rules project.
4. In the **Advanced Options**, under Visibility, select Specific Groups and type `rmpsp_landlord_resources`.

In this scenario, only users with access to landlord resources will be able to see the rules and the respective explanation in Case Manager.

info

With this option, you can create scenarios where a user can see an Alert/Decline but the explanation won't be available.

Rules precedenceâ□□â□□

An event can trigger several rules in multiple rules projects. It is important to understand what happens when an event triggers several rules that generate different outcomes. The following sections explain this precedence level for rules that generate decisions and alerts.

Precedence in decisionsâ□□â□□

The following image shows the precedence level for rules that trigger an approve or decline decision. The label *triggered* in this example means that at least one rule in the rules project has been triggered.

Rules precedence regarding decisions works as follows:

- **Scenario 1**: If no rule is triggered, the final outcome is â□□approveâ□□

- **Scenario 2**: When one of the rules with the decision decline is triggered, the final outcome is decline.
- **Scenarios 3 and 4**: When multiple rules with different decisions are triggered, the final outcome is the decision of the rule that has triggered first.

Precedence in alertsââ

The following image shows the precedence level for rules projects that trigger an alert:

Rules precedence regarding alerts works as follows:

- **Scenario 1**: If no rule is triggered, the transaction is not displayed in Case Manager's Alerts page. However, fraud fraud prevention agents can still access the transaction in Case Manager's search page.
- **Scenario 2**: When one of the rules with the action alert is triggered, the transaction is displayed in Case Manager's alert page. You have access to the reason explaining why the rule was triggered.
- **Scenario 3**: When multiple rules with alert action are triggered, the transaction is displayed in Case Manager's alert page. You have access to the reason explaining why each rule was triggered.

Rules orderââ

The order in which rules are processed can be defined inside each rules project. This allows you to define the order according to the priority of the outcome, i.e. rules with higher priority (e.g. hard decline) can be evaluated before rules with lower priority (e.g. âhard approveâ).

Rule componentsâ

Rules are composed of variables, conditions, and integrations with enrichment functions and environment metrics/variables. Learn about each of these components to understand how you can use them when building a rule.

Create rulesâ

A rule, when used in a workflow, evaluates if a certain condition is true for the current transaction. According to the result, the rule can trigger actions in the workflow like rejecting or accepting a transaction, and contribute to the score of the transaction.

The following sections help you successfully create rules for your business needs:

- Create rules.
- Operators in rules.
- Data types in rules.
- Default UDFs.

Request a reviewâ

Ensuring every change is reviewed by a second pair of eyes helps prevent unwanted or unplanned changes in production. As such, the Four Eyes Check feature allows you to request reviews from your colleagues.

Learn how to:

- Request peer reviews
- Review rules

Create listsâ

A list is a resource that allows you to store items (e.g. card IDs), which can be referenced in rules.

You can create lists in Pulse with different types of information. Use lists when writing rule conditions to confirm if the value of a given item is stored in a certain list. For example, you can create rules that decline or accept transactions performed by credit cards or digital events whose IDs are listed to be declined or approved.

In general, PosNeg lists are typically configured to identify items as "approve" or "decline" using a combination of key-value pair and activation timeframe.

Consider the following examples.

Decline listsâ

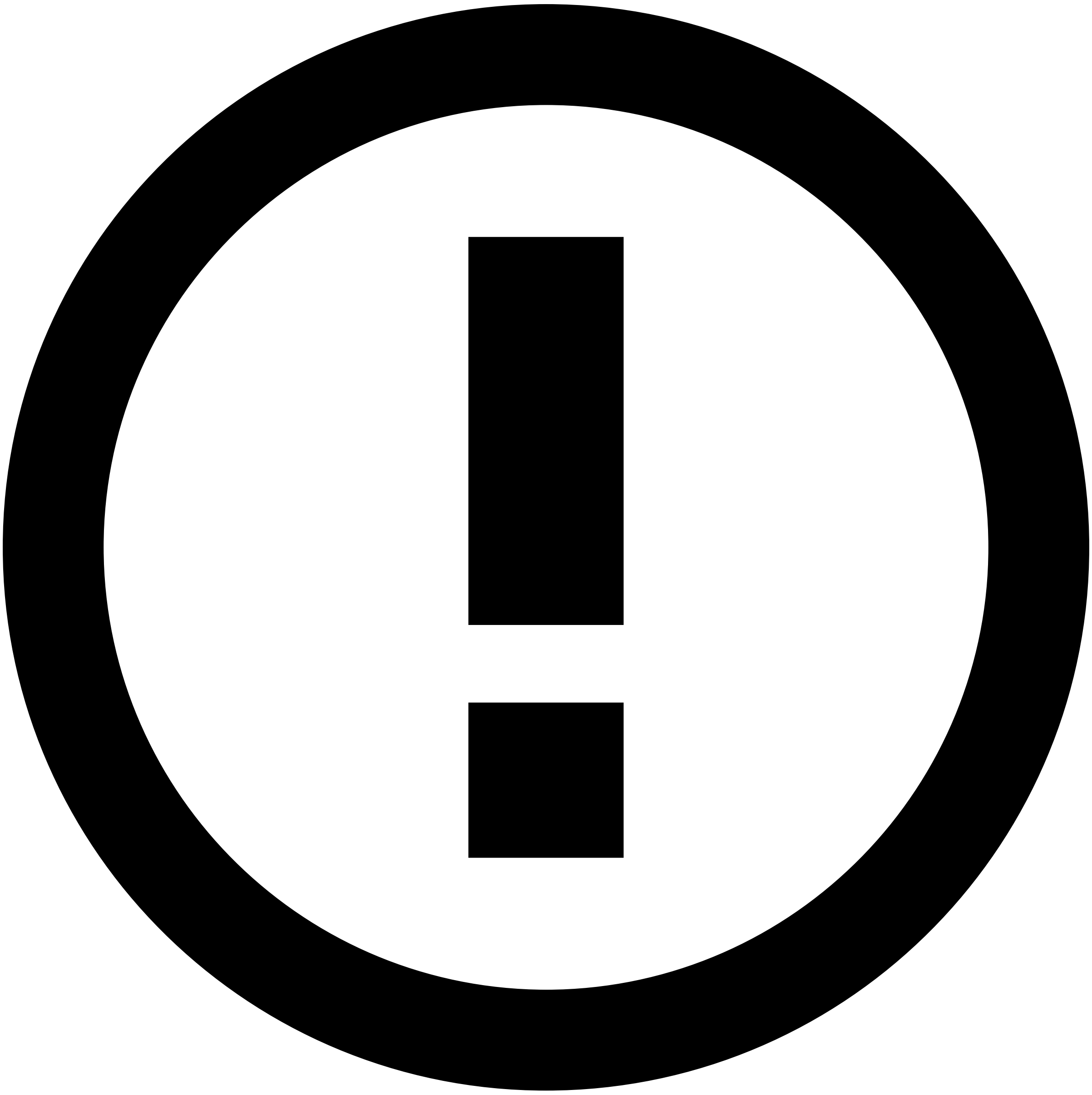
The following image illustrates an example of a card listed to be **declined**.

info

Timeframe: As a standard configuration, a card is permanently stored as listed to be declined. Therefore, the **Active from** option only specifies the initial date. However, in the case of a fraud feedback event (e.g. cardholder response via API or fraud prevention agent feedback via Case Manager) identifying the transaction as *Not Fraud*, the card is automatically tagged as `approve` (i.e. listed to be approved), thus overriding the `decline` value.

Approve listsâ

The following image illustrates an example of a card listed to be **approved**:



info

Timeframe: As a standard configuration, a card is listed to be approved within 2 hours (the time is configurable). After that, the card becomes automatically an inactive item (but still on the list). The **Active from** and **To** options are used to define this time interval.

To learn how to create lists, refer to the Pulse documentation - Creating Lists and Adding List Items.

Default listsâ

Transaction Fraud for PSPs includes default lists to speed up your work. Check the Lists page to confirm the out-of-the-box lists.

List management serviceâ

The list management service allows you to configure the automatic list transitions that an entity (e.g. card ID) undergoes when triggered by different rule actions.

Rule actions have standard values to identify decline and approve lists in a PosNeg List. The list management service is responsible for:

1. Checking when a rule triggers one of those actions.
2. Adding the entity to the PosNeg List and tagging it as decline or approve.

You can extend and adapt the list management service to fulfill your use cases.

To open the list management service:

1. In the sidebar, select the **Workflows** tab.
2. In the Diagram, select the pencil to edit the *Workflow List Manager*.

tip

Advanced functionality: Configurations in the list management service should be performed by advanced users to ensure that the transitions between lists work as expected. Note that, if the transitions are not well configured, the rules that depend on these transitions will not yield the targeted results.

How the list management service worksâ

The list management service is configured out of the box to classify card IDs as:

- Listed to be declined

• Listed to be approved

Decline listâ

The entries in the **Configuration** table with the *rules.0* prefix are associated with the same action (i.e. to list a card ID as declined). In this case, this configuration is set up to tag a card as **decline**:

Key	Value	Mandatory	Explanation
rules.0.actions	Auto Decline Card	Yes	The rule action that triggers this service to list a card ID to be declined.
rules.0.entity	payment_card_id	Yes	The entity that is added to the list (i.e. the key of the key-value pair in the list configuration options).
rules.0.listid	ffce7e5ce8f4013a	Yes	The list ID that is used to store the card ID (i.e. PosNeg Listed Cards).
rules.0.appid	c145d72d00d46bab	No	The app ID where the list exists. If not defined, the default is the app ID where the service exists.
rules. 0.listFieldName	value	No	The name of the list's field that is used in the list ID to store the card ID.
rules.0.entries. 0.value	decline	Yes	The value used to list a card ID to be declined (i.e. the value of the key-value pair in the list configuration options).

Approve listâ

The entries in the **Configuration** table with the prefix *rules.1* are used to list card IDs to be approved:

Key	Value	Mandatory	Explanation
rules.1.actions	Auto Approve Card	Yes	The rule action that triggers this service to list a Card ID to be approved.
rules.1.entity	payment_card_id	Yes	The entity that is added to the list (i.e. the key of the key-value pair in the list configuration options).
rules.1.listid	ffce7e5ce8f4013a	Yes	The list ID that is used to store the card ID (i.e. PosNeg Listed Cards).
rules.1.appid	c145d72d00d46bab	No	The app ID where the list exists. If not defined, the default is the app ID where the service exists.
rules. 1.listFieldName	value	No	The name of the list's field that is used in the list ID to store the card ID.
rules.1.entries. 0.value	approve	Yes	The value used to list a card ID to be approved (i.e. the value of the key-value pair in the list configuration options).
rules.1.entries.0.ttl	7200000	No	The duration that a card remains in the PosNeg Listed Cards list as approved in miliseconds.

Evaluate and test rulesâ

You can evaluate your rules to assess their effectiveness by creating and analyzing a rule snapshot evaluation.

Deploy rules to runtimeâ

The deployment process of rules is different depending on whether this is the first deployment, or an iteration over an existing deployment.

Learn more about this topic in Pulseâs Deploying Rules to the Production Environment page.